

# IMPACT OF SAMPLING FREQUENCY ON THE PERFORMANCE OF EDGE STREAM PROCESSING ARCHITECTURE IN IIOT SYSTEMS FOR AGV VEHICLES

Patryk Pięta\*<sup>1</sup>

<sup>1</sup>Faculty of Foundry Engineering, AGH University of Krakow, Krakow 30-059, Poland

*This paper investigates the performance limits and scalability of an edge computing node based on Apache Kafka and Apache Spark within Industrial Internet of Things systems. The continuous monitoring of Autonomous Guided Vehicles generates massive volumes of telemetry data that must be processed with minimal latency. Shifting the analytical workload to the edge of the network resolves cloud latency issues but introduces strict hardware constraints. The main objective of this study is to determine the saturation point and bottleneck triggers for a containerized, distributed stream processing architecture deployed in an edge environment. A comprehensive testbed was developed, featuring a kinematic emulator of a holonomic Mecanum drive vehicle that generates high-frequency telemetry data. Stress tests were conducted by incrementally increasing the data sampling and transmission frequency from 10 Hz to 500 Hz. The measured parameters included central processing unit utilization, random access memory consumption across individual microservices, and asynchronous stream processing latency. The experimental results identify the precise throughput threshold at which the edge node infrastructure saturates, revealing that the primary bottleneck is not CPU utilization, but rather I/O operations causing drastic latency spikes. These findings provide engineers with crucial metrics and architectural guidelines required for the accurate provisioning of computational resources before deploying real-time systems on the factory floor.*

**Keywords:** edge computing, AGV, stress testing, Apache Kafka, Apache Spark, Industry 4.0, IoT

**Acknowledgments:** This research was conducted as part of an engineering thesis at the AGH University of Krakow.

## 1 INTRODUCTION

The Fourth Industrial Revolution (Industry 4.0) relies heavily on the integration of cyber-physical production systems, including automated fleets of Autonomous Guided Vehicles (AGVs). To ensure the faultless operation of these machines and optimize intralogistics processes, the implementation of the Digital Twin concept is essential, requiring continuous, real-time monitoring of multidimensional telemetry parameters.

The increasing precision of IoT sensors and high sampling frequencies lead to the generation of massive data volumes within Industrial Internet of Things (IIoT) systems. The traditional Cloud Computing approach faces significant limitations in this context. These primarily include unpredictable network latency, the risk of packet loss, and high bandwidth costs, which often disqualify cloud-based solutions from critical, real-time analytical tasks.

Edge Computing addresses these challenges by shifting the analytical workload closer to the data source, directly onto the factory floor. Deploying distributed, containerized stream processing architectures based on technologies such as Apache Kafka and Apache Spark enables asynchronous data pipeline analysis with minimal delay. However, implementing this technology stack on edge computing nodes introduces strict hardware limitations that require precise provisioning of computational resources.

The primary objective of this paper is to evaluate the performance and identify the bottlenecks of a local stream processing architecture. This study presents the results of stress tests conducted on a cus-

tom testbed that simulates the kinematic and sensory telemetry of a holonomic Mecanum-drive AGV. The research investigates the impact of incrementally increasing data transmission frequencies on central processing unit utilization, random access memory consumption across individual microservices, disk/network I/O operations, and the overall throughput latency of the IIoT system.

## 2 MATERIALS AND METHODS

### 2.1 Materials and experimental setup

The experimental environment (testbed) was designed as an integrated edge computing node utilizing a microservices architecture based on Docker container virtualization. The tests were conducted on a workstation equipped with an Intel Core i7-13650HX processor and 16 GB of DDR5 RAM. The core of the data processing architecture was built upon an Apache Kafka cluster acting as an asynchronous message broker, and an Apache Spark engine responsible for real-time stream analytics. For persistent storage and aggregation of telemetry data, TimescaleDB, a database optimized for time-series workloads, was deployed.

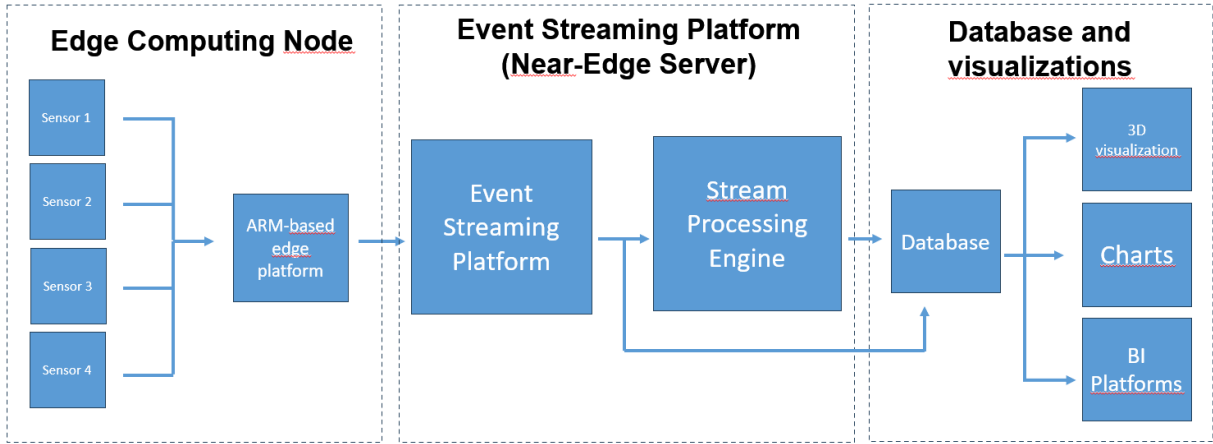


Figure 1: System architecture of the event-driven IoT pipeline.

### 2.2 Kinematic model and data generation

To ensure full control over the data generation frequency, the physical research object was replaced by a custom software emulator. This emulator replicates the kinematics of an AGV equipped with a four-wheel holonomic Mecanum drive. The linear and angular velocity vectors of the vehicle are determined based on the rotational speeds of individual wheels, described by the following forward kinematics equations:

$$v_{forward} = \frac{r}{4}(\omega_{FL} + \omega_{FR} + \omega_{RL} + \omega_{RR})$$

$$v_{strafe} = \frac{r}{4}(-\omega_{FL} + \omega_{FR} + \omega_{RL} - \omega_{RR})$$

$$\omega_{turn} = \frac{r}{4(l_x + l_y)}(-\omega_{FL} + \omega_{FR} - \omega_{RL} + \omega_{RR})$$

where  $r$  denotes the wheel radius,  $\omega$  is the angular velocity of the respective wheels (Front-Left, Front-Right, Rear-Left, Rear-Right), and  $l_x$  and  $l_y$  represent the distances from the wheels to the vehicle's

geometric center of gravity. Based on this model, the emulator generates multidimensional data payloads in JSON format. These payloads encompass power supply parameters (voltage, current), motor states (rotational speed, bearing temperature), and readings from a virtual IMU sensor (linear accelerations and angular velocities).

### 2.3 *Stress testing methodology*

The main phase of the experiment involved conducting a series of stress tests on the local edge node. The research scenario assumed an incremental increase in the transmission frequency of telemetry packets by the emulator, starting from an initial value of 10 Hz (the baseline AGV reporting frequency) until the system reached its saturation point (exceeding 500 Hz). During each testing interval, system resource utilization was profiled using tools integrated into the Docker engine.

### 2.4 *Stream processing latency and I/O saturation analysis*

To accurately assess the real-time capabilities of the architecture, the asynchronous stream processing latency was evaluated based on micro-batch metrics extracted directly from the Apache Spark UI. Because the Spark streaming job was configured with a fixed 1-second trigger interval, the total processing latency ( $L_{total}$ ) was formulated to account for queuing dynamics and potential I/O bottlenecks:

$$L_{total} = L_{queue} + L_{proc} + L_{I/O\_stall}$$

where  $L_{queue}$  represents the average time a message spends in the Kafka topic before the next micro-batch is triggered (approximated as 0.5 s for a 1-second window),  $L_{proc}$  is the actual micro-batch execution time (Batch Duration), and  $L_{I/O\_stall}$  accounts for any prolonged delays caused by disk or network I/O operations (Operation Duration), which serve as the primary indicator of system saturation.

### 2.5 *Queueing dynamics and sampling constraints*

A critical aspect of the methodology involves the interaction between the sampling frequency and the Spark micro-batch interval (set to 1 second). At high sampling frequencies (e.g., 500 Hz), the input rate significantly exceeds the consumer's processing capacity within a single batch window. Consequently, the observed message lag behaves as a stochastic process rather than a linear queue, where the collected metrics represent instantaneous "snapshots" of the buffer (ranging from 0 to 200 payloads). This non-deterministic queueing behavior confirms that at the 500 Hz threshold, the system transitions from continuous real-time processing to a burst-mode operation, characterized by high volatility in message latency.

### 2.6 *Research procedure*

To ensure the reproducibility of the results, the research process was conducted according to a standardized cycle, comprising five operational phases:

1. Initialization phase (Cold Start): Each measurement session began with a complete restart of the container environment to clear transient states in databases and Kafka buffers. Prior to measurements, the system was maintained in an Idle state for 60 seconds to stabilize the runtime environment.
2. Load generation phase: The edge emulator was set to operate at a defined packet transmission frequency (10, 50, 100, 250, and 500 Hz). Each scenario lasted 180 seconds, allowing for the collection of a statistically significant sample of telemetry data.

3. Data acquisition phase: During each session, system metrics were recorded at one-second intervals using the Docker stats interface. Data regarding CPU utilization (%), RAM allocation, and I/O network throughput were collected. Concurrently, performance logs were extracted from the Apache Spark UI to calculate  $L_{total}$ .
4. Aggregation phase (Post-processing): The collected raw logs were subjected to a normalization process, removing outliers resulting from transient host context switching. Data were then averaged for each load interval.
5. Validation phase: In the final stage of the cycle, the obtained resource utilization metrics were correlated with the processing latency to pinpoint the exact saturation trigger. The analysis verified whether the bottleneck originated from CPU computational limits, RAM exhaustion, or I/O network and disk throughput constraints within the Apache Spark engine.

### 3 RESULTS

The following section presents the empirical findings from the stress tests conducted on the edge computing node. The evaluation focuses on the scalability of computational resources, network throughput, and the total stream processing latency across five distinct data sampling frequencies: 10 Hz, 50 Hz, 100 Hz, 250 Hz, and 500 Hz.

#### 3.1 CPU utilization and computational scalability

The first metric evaluated was the Central Processing Unit (CPU) utilization across the core microservices. As illustrated in Fig. 1, the data consumer module, responsible for deserializing incoming Kafka payloads, exhibited a strictly linear increase in CPU load, scaling from 1.55% at 10 Hz to 29.65% at 500 Hz. The Apache Kafka broker showed a similar upward trend, reaching 21.56% under extreme load.

Conversely, the Apache Spark analytical engine presented a non-linear and counterintuitive behavior. At lower frequencies (10 Hz), it maintained a high processing footprint of approximately 34.49%. However, at the 500 Hz threshold, its average CPU utilization unexpectedly dropped to 16.37%. This anomaly indicates that the analytical engine was no longer computationally saturated but was instead hindered by external wait states.

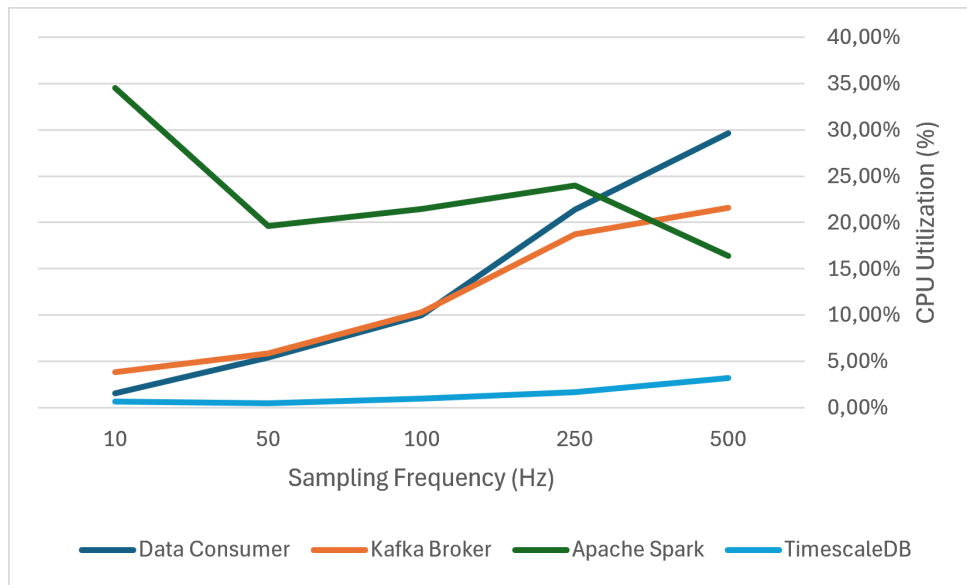


Figure 2: Average CPU Utilization Across Microservices vs. Sampling Frequency

### 3.2 Memory footprint stability

Unlike CPU utilization, the Random Access Memory (RAM) allocation remained highly static throughout the stress testing phases. The microservices utilizing the Java Virtual Machine (JVM) environment inherently reserved significant memory pools regardless of the actual data throughput. The Kafka broker consistently consumed approximately 1.1 GB of RAM, while the Apache Spark container hovered around 850 MB.

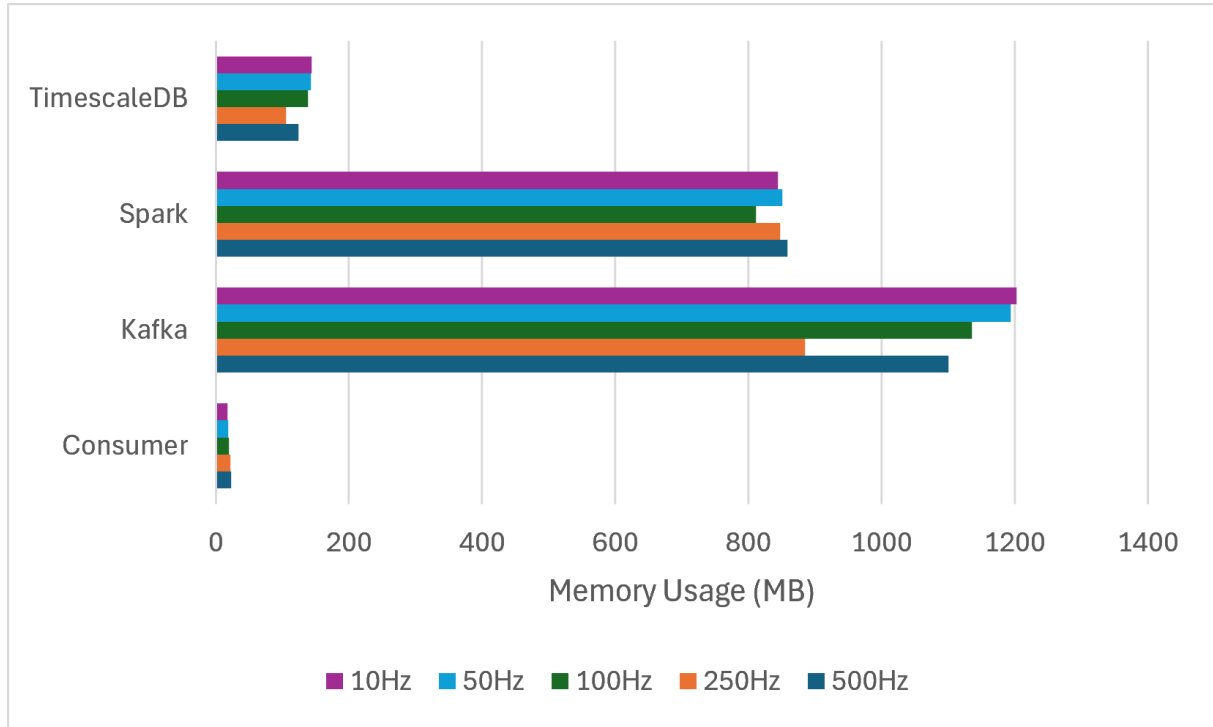


Figure 3: Random Access Memory (RAM) Allocation by Edge Node Components

By contrast, the Python-based data consumer and the TimescaleDB database proved to be highly lightweight, consuming around 20 MB and 140 MB, respectively. These results (Fig. 2) confirm that memory is not the dynamic bottleneck in this architecture, but it strictly dictates the baseline hardware requirements for deploying such a node on the factory floor.

### 3.3 Network throughput and analytical engine saturation

To explain the unexpected drop in Spark's CPU utilization at 500 Hz, the network ingress (NetIO) was correlated with the analytical processing load. As depicted in Fig. 3, the volume of data retrieved by the consumer from the Kafka broker increased exponentially, reaching over 64 MB of NetIO during the 500 Hz test cycle.

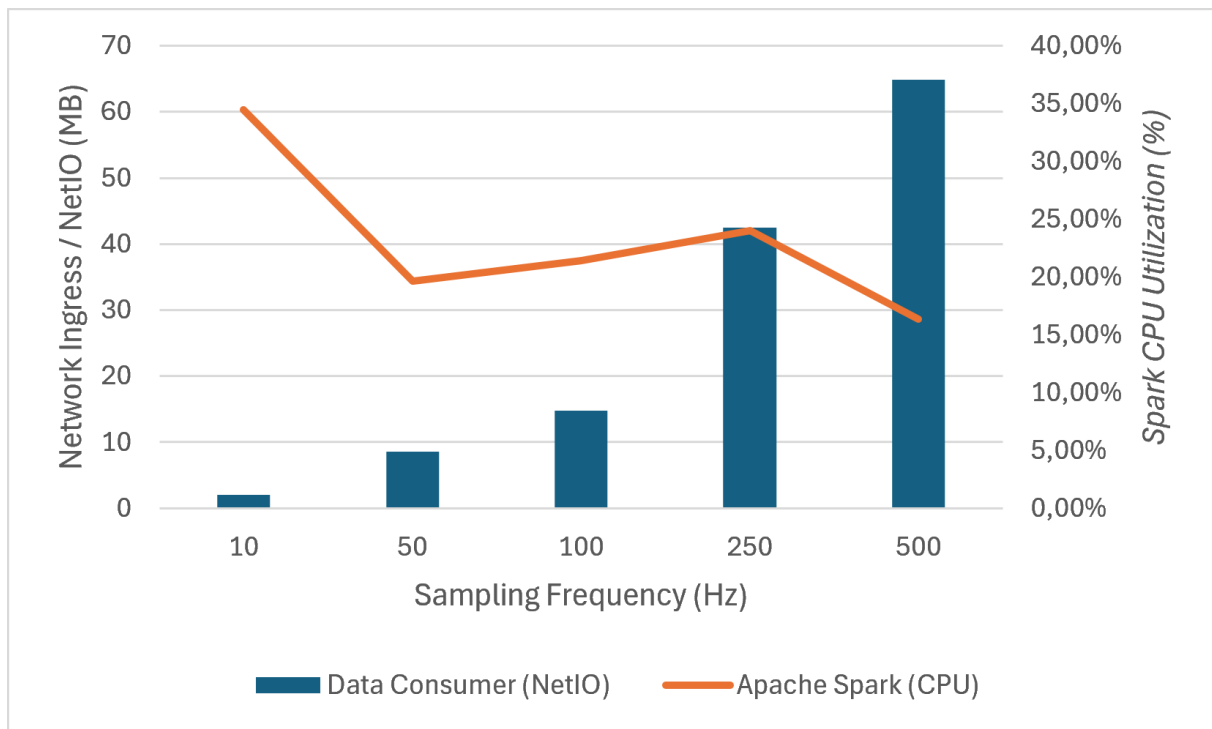


Figure 4: Correlation Between Network Ingress (Throughput) and Analytical Engine Processing Load

While the network layer successfully buffered and transmitted the massive data volume, the Spark engine failed to process it synchronously. The intersection of the rising NetIO bars and the falling Spark CPU line in Fig. 3 visually identifies the exact saturation point of the edge node. The system experienced severe backpressure, leading to resource starvation not at the CPU level, but at the input/output (I/O) level.

### 3.4 *Message broker queueing dynamics*

To further understand the behavior of the data pipeline under stress, the internal message lag within the Apache Kafka broker was monitored. The lag represents the number of unread telemetry payloads buffered in the topic queue before being ingested by the analytical engine. As shown in Fig. 5, the queueing dynamics exhibit a characteristic sawtooth pattern, reflecting the micro-batch processing nature of the consumer.

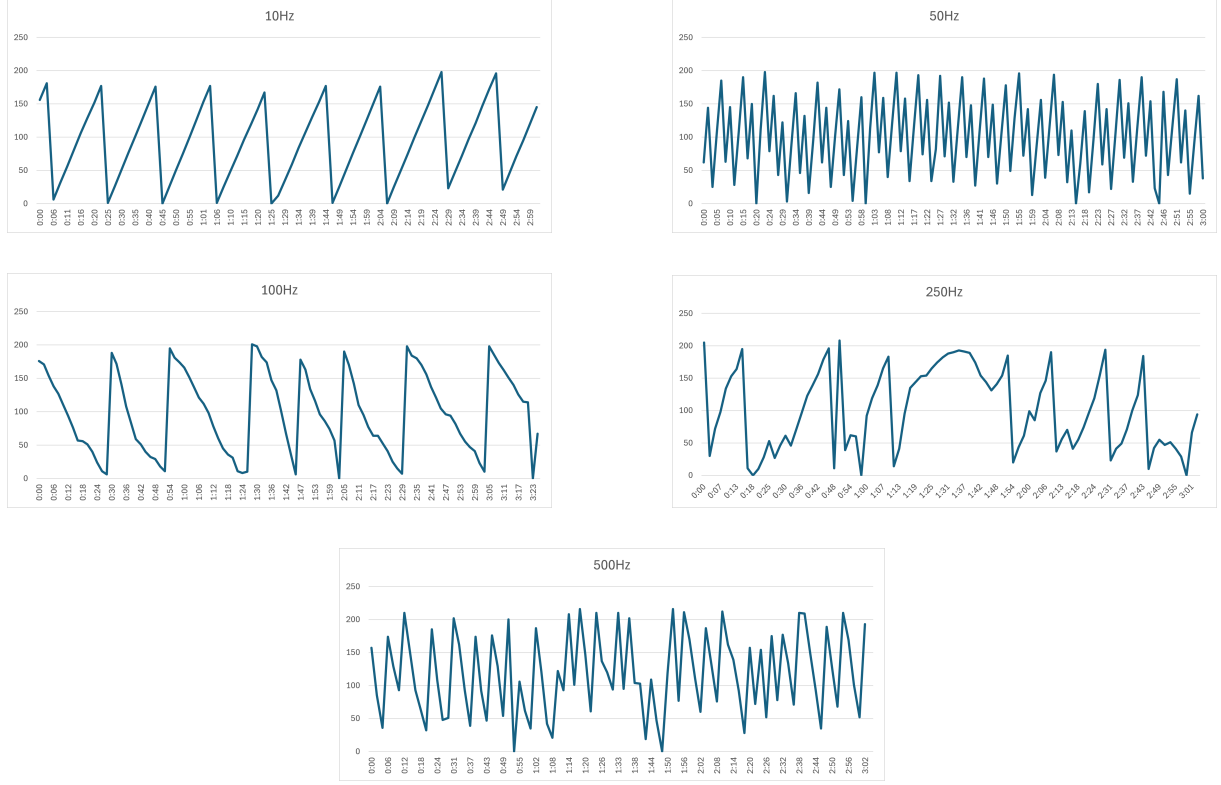


Figure 5: Kafka Broker Lag (unprocessed messages) over time across varying frequencies.

During low-to-medium frequency tests (10 Hz and 50 Hz), the lag demonstrates highly regular, periodic drops to exactly zero. This indicates that the consumer consistently and fully empties the buffer during each polling cycle without any data loss. However, as the sampling frequency escalates to 250 Hz and ultimately 500 Hz, the periodicity breaks down into a chaotic, dense pattern. The analytical engine becomes unable to clear the buffer entirely within a single window (reaching the implicit batch size limits of approximately 200 messages). This erratic queueing behavior serves as an early indicator of processing pipeline saturation, directly preceding the massive latency spikes observed in the final stage of the pipeline.

### 3.5 Stream processing latency and I/O bottleneck

The ultimate validation of the edge architecture is its ability to maintain real-time responsiveness. Table 1 and Fig. 4 present the total stream processing latency ( $L_{total}$ ) formulated in the methodology section. For sampling frequencies up to 250 Hz, the system demonstrated high stability, maintaining an average latency of approximately 0.75 seconds, comfortably below the strict 1.0-second real-time Service Level Agreement (SLA) threshold.

Table 1: Stream processing latency analysis across different sampling frequencies

| Frequency | Queue Delay ( $L_{queue}$ ) | Batch Duration ( $L_{proc}$ ) | Total Latency ( $L_{total}$ ) | System Status  |
|-----------|-----------------------------|-------------------------------|-------------------------------|----------------|
| 10 Hz     | 0.50 s                      | 0.30 s                        | 0.80 s                        | Real-time      |
| 50 Hz     | 0.50 s                      | 0.25 s                        | 0.75 s                        | Real-time      |
| 100 Hz    | 0.50 s                      | 0.25 s                        | 0.75 s                        | Real-time      |
| 250 Hz    | 0.50 s                      | 0.25 s                        | 0.75 s                        | Real-time      |
| 500 Hz    | 0.50 s                      | 0.35 s                        | > 10.00 s                     | I/O Saturation |

However, at 500 Hz, the architectural limits were breached. Performance logs extracted from the Spark UI revealed severe I/O operation stalls (WAL commits and disk writes) lasting upwards of 10 seconds. Consequently,  $L_{total}$  spiked drastically, entirely compromising the system's ability to trigger predictive maintenance alerts in time. This confirms that the ultimate bottleneck for containerized edge stream processing lies within the disk and network I/O throughput rather than raw computational power.

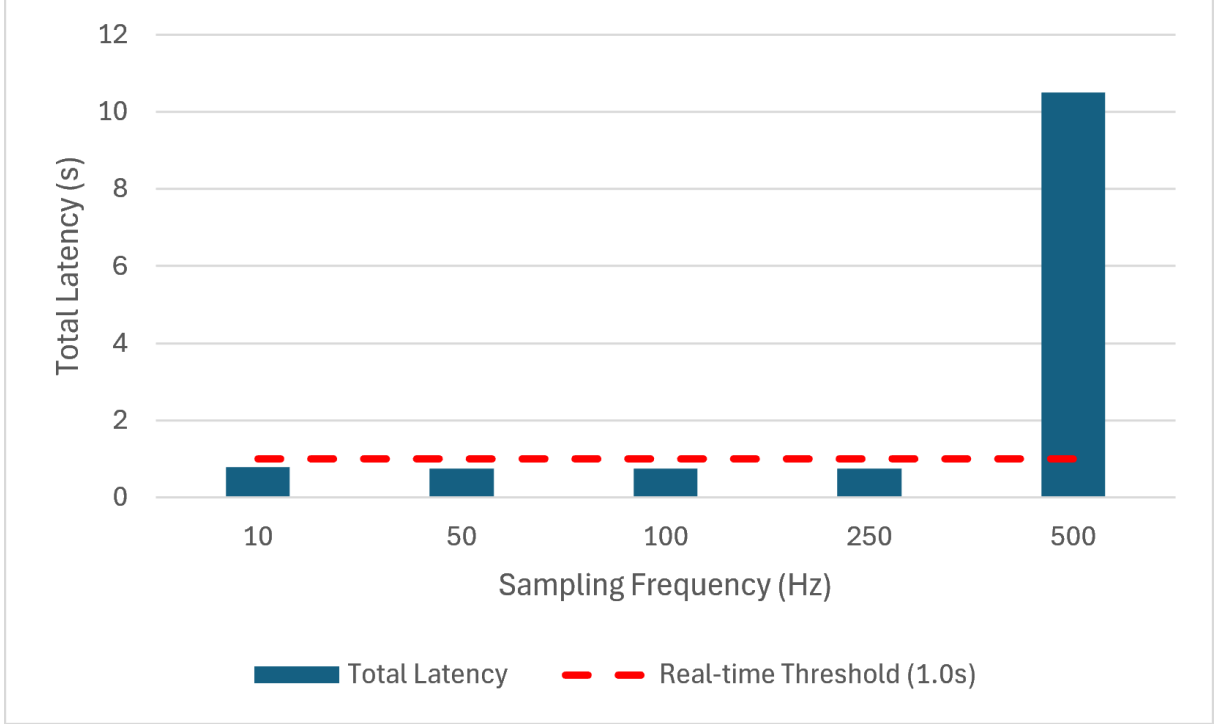


Figure 6: Total Stream Processing Latency with Real-Time SLA Constraint

## 4 DISCUSSION

The experimental results obtained from the stress tests provide a comprehensive insight into the performance limitations of containerized, distributed stream processing architectures deployed at the edge. The analysis identifies a clear transition in system bottlenecks as data sampling frequencies escalate.

### 4.1 Interpretation of performance bottlenecks

The observed non-linear behavior of the Apache Spark engine, specifically the paradox of decreasing CPU utilization coupled with rising latency at 500 Hz, provides a significant contribution to the understanding of edge node saturation. While initial assumptions might suggest that processing high-frequency telemetry data requires solely CPU power, our findings demonstrate that the system becomes I/O-bound well before computational capacity is exhausted. The "sawtooth" queueing dynamics in Kafka, observed in Fig. 2, confirm that the system initially operates in a steady state, but at 500 Hz, it enters a "burst-mode" operation where disk and network I/O commits (WAL writes in TimescaleDB) become the primary limiting factors. Specifically, at a 500 Hz sampling frequency, the analytical engine is forced to perform frequent, synchronous Write-Ahead Log (WAL) commits to the TimescaleDB storage layer; these blocking I/O operations exceed the write latency of the underlying disk subsystem, causing the stream processing engine to stall while awaiting storage confirmation. This identifies I/O throughput as the critical hardware requirement for real-time IIoT stream processing.

## 4.2 *Implications for real-time IIoT systems*

The study confirms that the proposed edge computing architecture is highly efficient for baseline AGV operations, maintaining stability and low latency (approx. 0.75 s) up to 250 Hz. However, the breach of the 1.0-second SLA at 500 Hz poses severe risks for intralogistics systems. In practical factory floor deployments, where latency is critical for safety-related collision avoidance or predictive maintenance, such "burst-mode" instability could lead to system failure. Engineers must therefore ensure that the provisioned hardware for edge nodes does not only focus on CPU cores but prioritizes high-throughput I/O storage solutions (e.g., NVMe SSDs) and optimized network stacks.

## 4.3 *Scalability and future perspectives*

While the current architecture proves scalable for single-vehicle diagnostics, the observed saturation point at 500 Hz highlights the limitations of the existing pipeline when managing fleet-wide telemetry. Future research directions will focus on several key architectural upgrades to enhance system robustness:

1. **Frontend Modernization:** Migration of the current interface to the Vue 3 framework will improve the modularity and responsiveness of the Digital Twin dashboard, allowing for more efficient rendering of multi-vehicle fleets.
2. **Simulation Environment:** Developing a dedicated simulation layer for edge-AI algorithms will enable stress-testing of complex swarm-intelligence behaviors without the need for high-frequency physical telemetry streaming.
3. **Advanced Machine Learning:** Future iterations will incorporate "deep" machine learning models directly into the Spark pipeline to replace simple threshold-based alerts with predictive failure analysis, leveraging the previously validated I/O headroom.

These improvements are essential for evolving the current prototype into a fully production-ready system capable of managing the complexities of Industry 4.0 intralogistics environments.

# 5 CONCLUSIONS

This research investigated the scalability and performance of a containerized edge computing architecture designed for real-time telemetry processing in intralogistics applications. Through a series of incremental stress tests ranging from 10 Hz to 500 Hz, we evaluated the system's responsiveness, resource utilization, and internal queueing dynamics.

The experimental results demonstrate that the proposed architecture is highly effective for standard AGV telemetry workloads, maintaining consistent real-time responsiveness with an average total latency of approximately 0.75 seconds—well within the 1.0-second SLA threshold—for frequencies up to 250 Hz.

However, our findings reveal a critical system saturation point at 500 Hz. Contrary to initial assumptions, this bottleneck was identified not as a computational failure of the CPU, but as a result of severe disk and network I/O throughput limitations within the analytical engine. The transition from continuous processing to "burst-mode" operation at higher frequencies, characterized by massive latency spikes, provides vital design guidelines for edge node deployment.

In conclusion, this study validates the feasibility of using microservices-based edge nodes for IIoT monitoring, while highlighting that for high-frequency telemetry, the provision of high-throughput storage and optimized I/O operations is as critical as computational power. These insights provide a robust

foundation for future implementations of digital twins and predictive maintenance systems in complex industrial environments.

## 6 ACKNOWLEDGMENTS

I am also grateful to the AGH University of Science and Technology for providing the research infrastructure necessary to complete this thesis. Special thanks go to the members of the Sensor Scientific Club for the collaborative spirit and engineering challenges that significantly contributed to my academic and professional development.

## 7 REFERENCES

### References

- [1] Apache Kafka. (2026). *Apache Kafka Documentation*. Available at: <https://kafka.apache.org/documentation/>
- [2] Apache Spark. (2026). *Structured Streaming Programming Guide*. Available at: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [3] TimescaleDB. (2026). *Time-series data and PostgreSQL*. Available at: <https://docs.timescale.com/>
- [4] Docker Inc. (2026). *Docker Documentation*. Available at: <https://docs.docker.com/>